

Instructor Guide

Python Programming Course — Module 2 Lab

Companion to the Module 2 Student Lab Guide

Purpose of This Guide

This guide supports the instructor or lab assistant running the Module 2 lab: numerical computation, floating-point precision, strings and f-strings, collections, nested data, the math module, basic kinematics and report generation. Students write small .py files and run them from the VS Code terminal (Command Prompt) inside the .venv they created in Module 1. It lists what to emphasise in each part, the errors that most commonly come up with their exact messages, and a checked answer key for every exercise.

Tip: Keep the Module 2 Student Lab Guide open next to this one — the part numbers below match it directly.

Before-Class Preparation

- Confirm every student finished the Module 1 checklist: python --version shows 3.13, .venv activates, and the VS Code terminal opens as Command Prompt. Students who did not will lose the first half of the lab.
 - Run the whole Student Lab Guide once on a lab laptop yourself, typing the code, as the course rule requires. Every expected output in the guide was produced with Python 3.13.15; outputs should be identical on any 3.13.x.
 - Have the finished lab files (module2 folder) on the shared drive as a reference, so you can diff a student's file against it quickly.
 - No internet access or package installs are needed during the lab — everything is in the standard library. Only Part 13 (git push) needs github.com.
 - Warn students in advance not to copy code from WhatsApp or other chat apps: curly quotes cause SyntaxError.
-

Suggested Time Allocation

The timetable has two 90-minute Module 2 labs (7 Oct and 8 Oct, 10:40–12:10). A realistic split for a class with mixed experience:

Session	Parts covered	Notes
Lab 1 · 10 min	Part 1 — module2 folder, activate .venv	Check every prompt shows (.venv) ... \module2> before moving on.
Lab 1 · 15 min	Parts 2–3 — numbers, engineering formulas	Short programs; good warm-up for typing accurately.
Lab 1 · 20 min	Part 4 — floating-point precision	Most important concept of the module; allow discussion.

Session	Parts covered	Notes
Lab 1 · 25 min	Parts 5–6 — strings, collections	Longest files to type; weaker typists will need the full time.
Lab 1 · 20 min	Exercises 2–6, commit and push	Finish with Part 13 so work is saved between labs.
Lab 2 · 15 min	Part 7 — nested data	Demonstrate the <code>[[0]*3]*3</code> trap on the projector.
Lab 2 · 15 min	Parts 8–9 — math module, string methods	Show students how to find a method in the docs themselves.
Lab 2 · 25 min	Part 10 — <code>physics.py</code> and <code>09_kinematics.py</code>	First time students import their own module.
Lab 2 · 20 min	Parts 11–12 — <code>report.py</code> , unit tests	Tests catch typos in <code>physics.py</code> and <code>report.py</code> .
Lab 2 · 15 min	Exercises 7–11, Parts 13–14	Walk the room for the final checklist.

What to Emphasise in Each Part

Part 2 — Numerical computation

`/` always gives a float, `//` floors towards minus infinity (`-7 // 2` is `-4`), and `**` binds tighter than unary minus (`-2 ** 2` is `-4`). `round()` uses round-half-to-even, which surprises students who expect `round(8.5)` to be `9`.

Part 3 — Engineering formulas

Insist on one variable per quantity with the unit in a comment. It is the habit that prevents most calculation errors in later engineering programs.

Part 4 — Floating-point precision

Stress that this is how binary floating point works in every language, not a Python bug. The practical rules are: never compare floats with `==`, use `math.isclose()`; use `math.fsum()` for long sums; use `Decimal` for money. The `3.13 round(2.675, 2) = 2.67` result is a good discussion point.

Part 5 — Strings

Strings are immutable; methods return new strings. The f-string specifier table is worth putting on the projector — Part 11 depends on it.

Parts 6–7 — Collections and nested data

Sets are unordered, which is why the program prints `sorted(set)`. For nested data, have students read `experiments["EXP-01"]["temps_C"][-1]` aloud from left to right.

Part 10 — Physics engine

Two ideas: a module is just a `.py` file you import, and a physics engine steps time forward instead of using closed-form formulas. The `dt` table shows the trade-off between accuracy and computation. The `dt = 0.1` result of exactly `2.0000 s` is itself a floating-point accumulation effect — link it back to Part 4.

Part 11 — Report generation

The column alignment comes entirely from the width and precision in each f-string field. Ask students to change one width and watch the table break.

Common Issues by Part

Every message below was produced by making the mistake deliberately with Python 3.13.15, so students will see exactly this last line.

Part 1 and every part — wrong folder

Symptom: python 01_numbers.py prints: python: can't open file '...\PythonCourse\01_numbers.py': [Errno 2] No such file or directory

Likely cause: The terminal is in PythonCourse (or a new terminal was opened) rather than module2, or the file was saved somewhere else.

Fix: Check the prompt ends in module2>. If not: .venv\Scripts\activate from PythonCourse, then cd module2. In the Explorer, confirm the file sits inside the module2 folder.

Part 1 — activation typed with forward slashes

Symptom: operable program or batch file. — for example after typing .venv/Scripts/activate.

Likely cause: Command Prompt needs backslashes in this command.

Fix: Type .venv\Scripts\activate exactly as in the guide.

Any part — pasted code

Symptom: SyntaxError: invalid character "'" (U+201C)

Likely cause: Code was copied from a chat app or document that replaced straight quotes with curly ones.

Fix: Retype the quotes in VS Code. The error shows the exact line.

Parts 4 and later — indentation

Symptom: IndentationError: expected an indented block after 'for' statement on line 2

Likely cause: The body of a for / if block was not indented.

Fix: Indent the lines inside the block by 4 spaces (VS Code inserts them with Tab).

Any part — typing mistakes in names

Symptom: NameError: name 'resistence' is not defined. Did you mean: 'resistance'?

Likely cause: A variable is spelled differently where it is used.

Fix: Python 3.13 suggests the right name (Did you mean ...?) — point students to it.

Part 5 — converting text to numbers

Symptom: ValueError: could not convert string to float: '351,27'

Likely cause: The text uses a comma as the decimal separator, or still contains other characters.

Fix: Clean the text first, for example `value.replace(",", ".")`, then call `float()`.

Part 10 — importing physics

Symptom: `ModuleNotFoundError: No module named 'physics'`

Likely cause: `physics.py` is not in the same folder as `09_kinematics.py`, or Windows saved it as `physics.py.txt` (hidden file extensions), or it was named differently.

Fix: In the VS Code Explorer, check the exact name and folder. Rename to `physics.py` if needed.

Watch for: Students who edit `physics.py` while `09_kinematics.py` is open sometimes forget to save `physics.py` (the tab shows a dot). The old version runs until they save.

Part 12 — unit tests

Symptom: `python -m unittest -v test_module2` ends with `FAILED (errors=1)` and `ModuleNotFoundError: No module named 'test_module2'`.

Likely cause: The tests were run from `PythonCourse` instead of `module2`.

Fix: `cd module2` and run the command again. If a test reports `FAIL` instead, the named function in `physics.py` or `report.py` has a typo — compare it with the guide.

Any part — division by zero

Symptom: `ZeroDivisionError: division by zero`

Likely cause: A student changed an input (for example resistance) to 0 while experimenting.

Fix: A good teaching moment: formulas need valid inputs. Change the value back.

Exercise Answer Key

Each answer below was run with Python 3.13.15; the output shown is the real output. Accept any correct approach — these are reference answers, not the only answers.

Exercise 2 (Part 2) — rope

```
rope_cm = 1000
piece_cm = 37
print("full pieces:", rope_cm // piece_cm)
print("left over:", rope_cm % piece_cm, "cm")
```

Output:

```
full pieces: 27
left over: 1 cm
```

Exercise 3 (Part 3) — kinetic energy

```
mass = 1200          # kg
speed_kmh = 90       # km/h
speed = speed_kmh * 1000 / 3600  # m/s
```

```
energy = 0.5 * mass * speed ** 2 # J
print(f"speed = {speed} m/s, kinetic energy = {energy / 1000:.1f} kJ")
```

Output:

```
speed = 25.0 m/s, kinetic energy = 375.0 kJ
```

Exercise 4 (Part 4) — 0.1 * 3

```
import math
print("0.1 * 3 == 0.3 ->", 0.1 * 3 == 0.3)
print("math.isclose(0.1 * 3, 0.3) ->", math.isclose(0.1 * 3, 0.3))
print("0.1 * 3 =", 0.1 * 3)
```

Output:

```
0.1 * 3 == 0.3 -> False
math.isclose(0.1 * 3, 0.3) -> True
0.1 * 3 = 0.30000000000000004
```

Exercise 5 (Part 5) — clean a line

```
raw = " p-07 ; inlet pressure;101.337 ;KPA "
tag, name, value, unit = [p.strip() for p in raw.split(";")]
unit = unit.lower().replace("kpa", "kPa")
print(f"{tag.upper()} {name.title()} = {float(value):.2f} {unit}")
```

Output:

```
P-07 Inlet Pressure = 101.34 kPa
```

Exercise 6 (Part 6) — °C to °F

```
readings = [23.1, 22.8, 24.5, 23.9, 25.2, 21.7, 22.0]
fahrenheit = [round(c * 9 / 5 + 32, 1) for c in readings]
print(fahrenheit)
```

Output:

```
[73.6, 73.0, 76.1, 75.0, 77.4, 71.1, 71.6]
```

Exercise 7 (Part 7) — nested data

```
experiments = {
    "EXP-01": {"material": "steel", "temps_C": [20, 150, 300], "passed": True},
    "EXP-02": {"material": "aluminium", "temps_C": [20, 90, 120], "passed": False},
    "EXP-03": {"material": "copper", "temps_C": [20, 60], "passed": True},
}
experiments["EXP-01"]["operator"] = "Your Name"
print(experiments["EXP-01"])
for exp_id, info in experiments.items():
    if info["passed"]:
        temps = info["temps_C"]
        print(f"{exp_id}: average {sum(temps) / len(temps):.1f} C")
```

Output:

```
{'material': 'steel', 'temps_C': [20, 150, 300], 'passed': True, 'operator': 'Your Name'}  
EXP-01: average 156.7 C  
EXP-03: average 40.0 C
```

Exercise 8 (Part 8) — atan2

```
import math  
print(math.atan2.__doc__)  
print("angle of (-1, 1):", math.degrees(math.atan2(1, -1)), "degrees")
```

Output:

```
Return the arc tangent (measured in radians) of y/x.  
  
Unlike atan(y/x), the signs of both x and y are considered.  
angle of (-1, 1): 135.0 degrees
```

Exercise 9 (Part 9) — two more string methods

```
print("'key=value'.partition('=') ->", "key=value".partition("="))  
print("'T-101'.removeprefix('T-') ->", "T-101".removeprefix("T-"))
```

Output:

```
'key=value'.partition('=') -> ('key', '=', 'value')  
'T-101'.removeprefix('T-') -> 101
```

Exercise 10 (Part 10) — Earth vs Moon

```
import physics  
earth = physics.projectile(20, 45)  
moon = physics.projectile(20, 45, g=1.62)  
print(f"Earth range at 45 deg: {earth['range']:.2f} m")  
print(f"Moon range at 45 deg: {moon['range']:.2f} m")  
print(f"Moon / Earth: {moon['range'] / earth['range']:.2f} times further")
```

Output:

```
Earth range at 45 deg: 40.77 m  
Moon range at 45 deg: 246.91 m  
Moon / Earth: 6.06 times further
```

Exercise 11 (Part 11) — report with 3 decimals

```
import report  
raw_data = {  
    "Temperature": {"unit": "C", "values": [21.43, 21.97, 22.61, 23.08, 22.40]},  
    "Pressure": {"unit": "kPa", "values": [101.3, 101.1, 100.9, 101.4, 101.2]},  
    "Flow rate": {"unit": "L/min", "values": [12.5, 12.9, 13.4, 12.2, 12.8]},  
    "Voltage": {"unit": "V", "values": [229.8, 230.4, 231.1, 229.5, 230.2]},  
}  
print(report.format_report("Cooling Loop Test - Run 7", raw_data, decimals=3))
```

Output:

Cooling Loop Test - Run 7						
=====						
Quantity	Unit	n	Mean	Min	Max	Std

Temperature	C	5	22.298	21.430	23.080	0.562
Pressure	kPa	5	101.180	100.900	101.400	0.172
Flow rate	L/min	5	12.760	12.200	13.400	0.403
Voltage	V	5	230.200	229.500	231.100	0.548

4 quantities, 20 readings in total.						

Note: Exercise 4: $0.1 * 3 == 0.3$ is False, for the same reason as Part 4. Exercise 9 has many valid answers; check that both examples run and use methods not in the table.

Things to Watch For While Students Are Working

- Glance at each prompt: it should start with (.venv) and end in module2>. Most problems in this lab are the wrong folder or an inactive .venv.
- Watch for unsaved files (a dot on the VS Code tab) — students run the old version and think their change did not work.
- Encourage students to read the last line of an error message first; Python 3.13's messages usually name the problem and the line.
- Early finishers can extend 10_report.py (a new quantity, a different title, more decimals) or add a new kinematics function with a test.
- Do a visible check at Part 14: ask each student to show Ran 9 tests ... OK and the pushed commit on GitHub before they leave.

Quick-Reference Troubleshooting Table

Symptom	Likely cause	Fix
can't open file ... No such file or directory	Terminal not in module2	Activate .venv, cd module2
'.venv' is not recognized ...	Forward slashes in activate	Type .venv\Scripts\activate
SyntaxError: invalid character "'" (U+201C)	Curly quotes from pasted code	Retype the quotes
IndentationError: expected an indented block ...	Block body not indented	Indent 4 spaces
NameError: name '...' is not defined. Did you mean ...?	Spelling mistake	Use the suggested name
ValueError: could not convert string to float	Comma or extra characters in the text	Clean the text before float()

Symptom	Likely cause	Fix
ModuleNotFoundError: No module named 'physics'	physics.py missing, misnamed or in another folder	Check name and folder in Explorer
No module named 'test_module2' (unittest)	Tests run from the wrong folder	cd module2 and rerun
ZeroDivisionError: division by zero	Input changed to 0	Restore a valid input
Output differs from the guide	Unsaved file or a typo in the code	Save; compare line by line with the guide

After the Session

- Check the GitHub commits: every student should have a module2 folder with ten lab files, physics.py, report.py, test_module2.py and their exercise files.
- Note which students did not reach Part 12 and follow up before Module 3 — Module 3 assumes they can write and import their own module.
- Record any issue that affected several students so the Student Lab Guide can be improved for the next cohort.

Draft v0.1 for instructor authoring and review (Content Development Workflow, Step 4). All code, answers and error messages were run on Windows 10 with Python 3.13.15 in a .venv from Command Prompt on 26 Sep 2026.